CHAP-5_BAYESIAN

Tawhid Tasdique

2026-01-30

Ex-5.1

Ex-5.1(a)

```
school1 <- read.table(file = "school1.dat",
                      quote = "/",
                      comment.char = "")

school2 <- read.table(file = "school2.dat",
                      quote = "/",
                      comment.char= "")

school3 <- read.table(file = "school3.dat",
                      quote = "/",
                      comment.char= "")

school1 <- school1 %>%
  dplyr::select(V1) %>%
  as.matrix() %>%
  as.numeric()

school2 <- school2 %>%
  dplyr::select(V1) %>%
  as.matrix() %>%
  as.numeric()

school3 <- school3 %>%
  dplyr::select(V1) %>%
  as.matrix() %>%
  as.numeric()
```

Theory

The kernel part derived in **HomeCopy-II** is non-standard or unfamiliar. So, we can't sample from it directly. Instead we will implement Monte-Carlo simulation to approximate $(\theta|y_1, ..., y_n)$.

First, we directly sample σ^2 from it's marginal posterior and then implement reuse those values to sample θ from its full conditional distribution i.e. $(\theta|y_1, ..., y_n)$.

We use the obtained Monte-Carlo samples to compute necessary means and 95% confidence intervals. Since, posterior distributions are not purely theoretical quantities, we will compute (empirical) quantile-based confidence intervals from the produced samples.

Note that, the theoretical mean of the full conditional distribution, $(\theta|\sigma^2, y_1, ..., y_n)$, is **NOT** the posterior mean for θ. The "posterior mean" refers to the **marginal** posterior's mean. But that is an unrecognized distribution, at least for θ it is. Hence, we have to approximate it by averaging the relevant Monte-Carlo samples for each school.

```
# Hyper-parameters
mu0 <- 5; sigma20 <- 4; k0 <- 1; v0 <- 2

# sample means
y1bar <- school1 %>% mean()
y2bar <- school2 %>% mean()
y3bar <- school3 %>% mean()

# sample sizes
n1 <- school1 %>% length()
n2 <- school2 %>% length()
n3 <- school3 %>% length()

# posterior parameters

kn1 <- k0 + n1
kn2 <- k0 + n2
kn3 <- k0 + n3

vn1 <- v0 + n1
vn2 <- v0 + n2
vn3 <- v0 + n3

################################################################

# sampling from 1st inverse-Gamma posterior
`sigma1^2|data` <- 1 / rgamma(
  n = 1000,
  shape = vn1 / 2,
  rate = (vn1/2)*(
    v0*(sigma20^2) +
    (n1-1)*var(school1) +
    (k0*n1/kn1)*(y1bar-mu0)
  )
)

# sampling from 2nd inverse-Gamma posterior
`sigma2^2|data` <- 1 / rgamma(
  n = 1000,
  shape = vn2 / 2,
  rate = (vn2/2)*(
    v0*(sigma20^2) +
    (n2-1)*var(school2) +
    (k0*n2/kn2)*(y2bar-mu0)
  )
)
```

```r
# sampling from 3rd inverse-Gamma posterior
`sigma3^2|data` <- 1 / rgamma(
  n = 1000,
  shape = vn3 / 2,
  rate = (vn3/2)*(
    v0*(sigma20^2) +
    (n3-1)*var(school3) +
    (k0*n3/kn3)*(y3bar-mu0)
  )
)

##############################################################

# theoretical means from full conditional distribution of theta
mu1 <- weighted.mean(x = c(mu0, y1bar), # school 1
                     w = c(k0, n1))
mu2 <- weighted.mean(x = c(mu0, y2bar), # school 2
                     w = c(k0, n2))
mu3 <- weighted.mean(x = c(mu0, y3bar), # school 3
                     w = c(k0, n3))

##############################################################

# sampling theta from 1st full conditional posterior
`theta|sigma1^2, data` <- sapply(
  X = `sigma1^2|data`,
  FUN = function(sig){
    rnorm(n = 1, mean = mu1, sd = sig)
  }
)

# sampling theta from 2nd full conditional posterior
`theta|sigma2^2, data` <- sapply(
  X = `sigma2^2|data`,
  FUN = function(sig){
    rnorm(n = 1, mean = mu2, sd = sig)
  }
)

# sampling theta from 2nd full conditional posterior
`theta|sigma3^2, data` <- sapply(
  X = `sigma3^2|data`,
  FUN = function(sig){
    rnorm(n = 1, mean = mu3, sd = sig)
  }
)

##############################################################

# posterior means approximated for theta
post.mean.theta.1 <- mean(`theta|sigma1^2, data`)
post.mean.theta.2 <- mean(`theta|sigma2^2, data`)
post.mean.theta.3 <- mean(`theta|sigma3^2, data`)
```

```r
# posterior means approximated for sigma
post.mean.sigma.1 <- mean(sqrt(`sigma1^2|data`))
post.mean.sigma.2 <- mean(sqrt(`sigma2^2|data`))
post.mean.sigma.3 <- mean(sqrt(`sigma3^2|data`))

##############################################################

# quantile-based 95% confidence interval for theta
post.ci.theta.1 <- quantile(x = `theta|sigma1^2, data`,
                     probs = c(0.025, 0.975))
post.ci.theta.2 <- quantile(x = `theta|sigma2^2, data`,
                     probs = c(0.025, 0.975))
post.ci.theta.3 <- quantile(x = `theta|sigma3^2, data`,
                     probs = c(0.025, 0.975))

# quantile-based 95% confidence interval for sigma
post.ci.sigma.1 <- quantile(x = `sigma1^2|data`,
                            probs = c(0.025, 0.975)) %>% sqrt()
post.ci.sigma.2 <- quantile(x = `sigma2^2|data`,
                            probs = c(0.025, 0.975)) %>% sqrt()
post.ci.sigma.3 <- quantile(x = `sigma3^2|data`,
                            probs = c(0.025, 0.975)) %>% sqrt()
```

Ex-5.1(b)

```r
theta1 <- `theta|sigma1^2, data`
theta2 <- `theta|sigma2^2, data`
theta3 <- `theta|sigma3^2, data`

(abc <- mean(theta1 < theta2 & theta2 < theta3))
```

```
## [1] 0.127
```

```r
(acb <- mean(theta1 < theta3 & theta3 < theta2))
```

```
## [1] 0.195
```

```r
(bac <- mean(theta2 < theta1 & theta1 < theta3))
```

```
## [1] 0.177
```

```r
(bca <- mean(theta2 < theta3 & theta3 < theta1))
```

```
## [1] 0.203
```

```r
(cab <- mean(theta3 < theta1 & theta1 < theta2))
```

```
## [1] 0.146
```

```r
(cba <- mean(theta3 < theta2 & theta2 < theta1))
```

```
## [1] 0.152
```

Ex-5.1(c)

Now, we need to simulate data from the posterior predictive distributions. The form of the posterior predictive distribution is too complicated and time-consuming to derive if not impossible. So, we're going to employ Monte-Carlo simulation to approximate it instead.

```r
# sampling from school1's posterior predictive distribution
y1pred <- sapply(
  X = 1:length(`sigma1^2|data`),
  FUN = function(i){
    rnorm(n = 1,
          mean = `theta|sigma1^2, data`[i],
          sd = `sigma1^2|data`[i])
  }
)

# sampling from school2's posterior predictive distribution
y2pred <- sapply(
  X = 1:length(`sigma2^2|data`),
  FUN = function(i){
    rnorm(n = 1,
          mean = `theta|sigma2^2, data`[i],
          sd = `sigma2^2|data`[i])
  }
)

# sampling from school3's posterior predictive distribution
y3pred <- sapply(
  X = 1:length(`sigma3^2|data`),
  FUN = function(i){
    rnorm(n = 1,
          mean = `theta|sigma3^2, data`[i],
          sd = `sigma3^2|data`[i])
  }
)

(abc <- mean(y1pred < y2pred & y2pred < y3pred))
```

```
## [1] 0.151
```

```r
(acb <- mean(y1pred < y3pred & y3pred < y2pred))
```

```
## [1] 0.178
```

```r
(bac <- mean(y2pred < y1pred & y1pred < y3pred))
```

```
## [1] 0.183
```

```r
(bca <- mean(y2pred < y3pred & y3pred < y1pred))
```

```
## [1] 0.187
```

```r
(cab <- mean(y3pred < y1pred & y1pred < y2pred))
```

```
## [1] 0.161
```

```r
(cba <- mean(y3pred < y2pred & y2pred < y1pred))
```

```
## [1] 0.14
```

Ex-5.1(d)

```r
(ans1 <- mean(theta1 > theta2 & theta1 > theta3))
```

```
## [1] 0.355
```

```r
(ans2 <- mean(y1pred > y2pred & y1pred > y3pred))
```

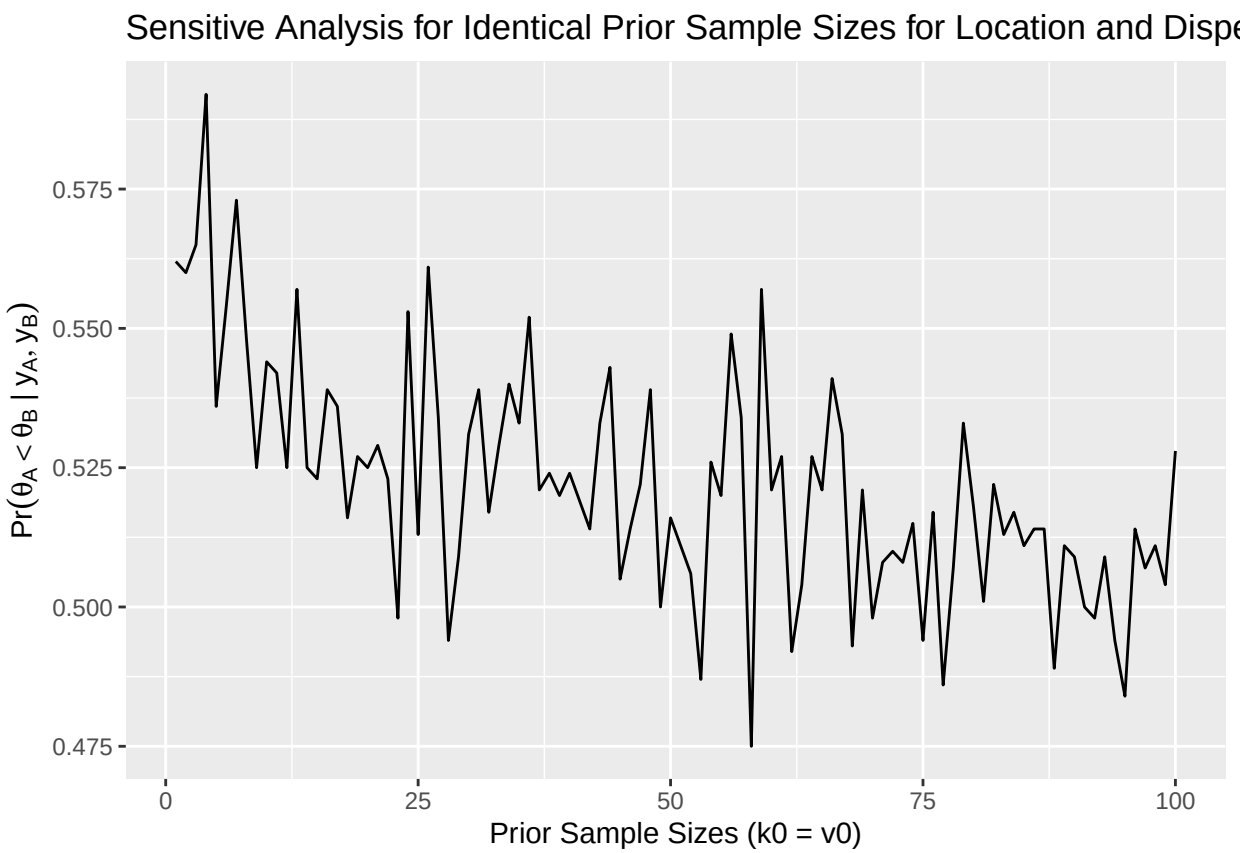
```
## [1] 0.327
```

Ex-5.2

What is sensitivity analysis? Explain.

```r
fun5.2 <- function(n = 16, # sample size
                   ybar, # sample mean
                   s, # sample standard deviation
                   mu0 = 75, # prior mean for location parameter
                   sigma20 = 100, # prior variance estimate
                   k0, # prior sample size for location parameter
                   v0, # prior sample size for variance parameter
                   nsim = 1000) { # size of simulation

  # posterior sample sizes
  kn <- k0 + n
  vn <- v0 + n

  # posterior mean
  mun <- (k0*mu0 + n*ybar)/kn

  # posterior variance
  sigma2n <- (v0*sigma20 + (n-1)*s^2 + k0*n*((ybar - mu0)^2)/kn)/vn

  # sampling from marginal posterior of variance parameter
```

```r
  `sigma^2 | data` <- 1/rgamma(n = nsim,
                               shape = (vn/2),
                               rate = (vn*sigma2n)/2)

  # sampling from full conditional distribution of location parameter
  `theta | sigma^2, data` <- sapply(
    X = `sigma^2 | data`,
    FUN = function(var.param) {
      rnorm(n = 1,
            mean = mun,
            sd = sqrt(var.param))
    }
  ) # Monte-Carlo simulation ends here.

} # function ends here

# range of prior sample sizes
prior.size <- 1:100

set.seed(27)
# generating posterior probabilities for the desired inequality surrounding location parameters of grou
`Pr(thetaA < thetaB | dataA, dataB` <- sapply(
  X = prior.size,
  FUN = function(size) {

    # prior sample sizes
    k <- v <- size

    thetaA <- fun5.2(ybar = 75.2, s = 7.3, k0 = k, v0 = v)
    thetaB <- fun5.2(ybar = 77.5, s = 8.1, k0 = k, v0 = v)

    return(mean(thetaA < thetaB))
  }
)

# line plot
tibble(
  `k0 = v0` = prior.size,
  prob.event = `Pr(thetaA < thetaB | dataA, dataB`
) %>%
  ggplot(aes(x = `k0 = v0`, y = prob.event)) +
  geom_line() +
  xlab(label = "Prior Sample Sizes (k0 = v0)") +
  ylab(label = quote(Pr(theta[A] < theta[B] ~ "|" ~ y[A], y[B]))) +
  labs(title = "Sensitive Analysis for Identical Prior Sample Sizes for Location and Dispersion Paramet
```



Conclusion

As can be seen, the probability drops as sample size increases. If you want you can take a larger range of sample sizes to observe if the results are consistent in the long run.

Ex-5.3

I've already derived all the distributions in **Assignment-4** and **HomeCopy-3**. **Assignment-4** can be found on **HomeCopy-1** and a part of it can also be found on **HomeCopy-3**. The marginal posterior distribution for θ is a non-standard distribution. So, it will be approximated via Monte-Carlo sampling.

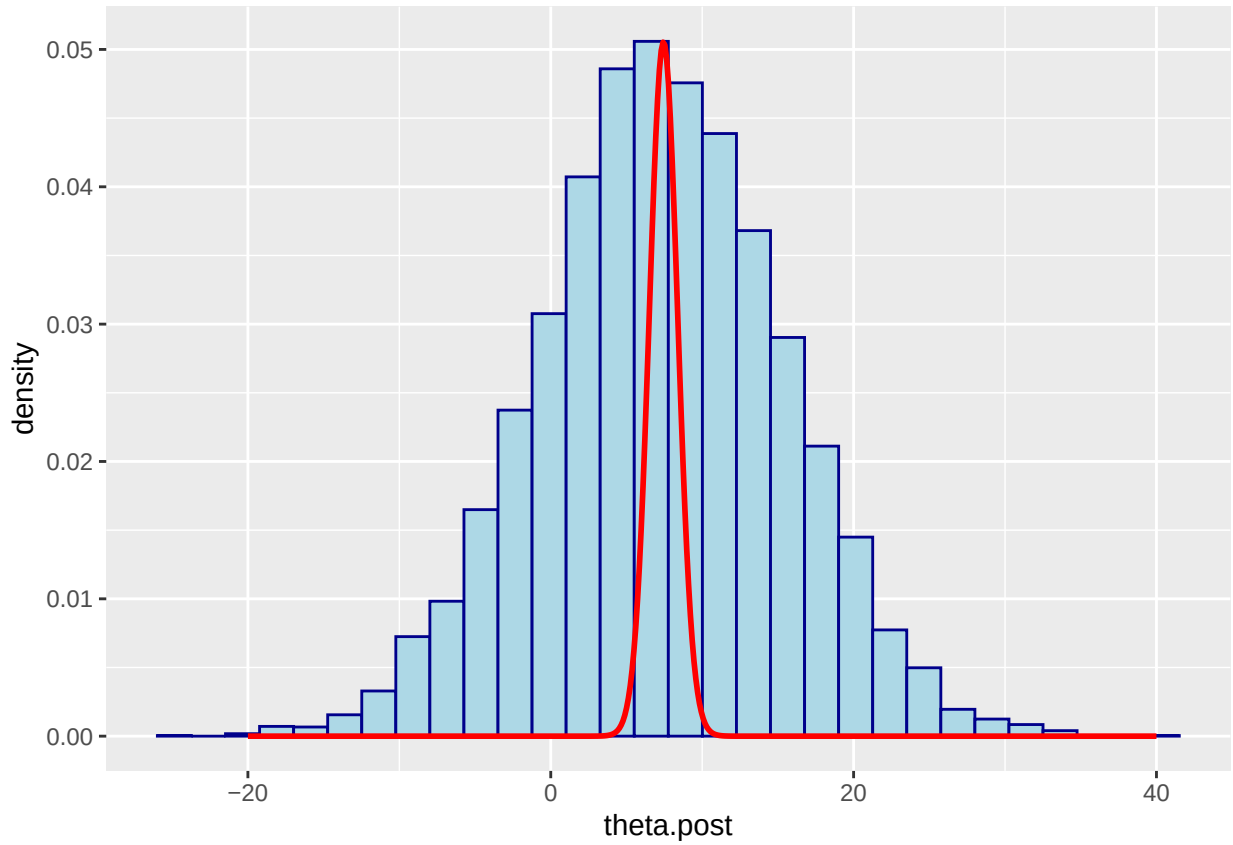
Then I will compare it with closely-spaced densities computed by a user-defined kernel of its PDF (probability density function).

```
set.seed(27)

# kernel part of marginal posterior of theta
# this function needs to be modified to abide by total law of probability
# I'm going to try discrete approximation and see what happens
mp.theta <- function(theta, k0 = 20, mu0 = 5.64, v0 = 7, sigma20 = 27.43, y){

  # data-related values
  n <- length(y)
  ybar <- mean(y)
```

```r
  # posterior parameters
  kn <- k0 + n
  vn <- v0 + n
  mun <- (k0*mu0 + n*ybar)/kn
  sigma2n <- (v0*sigma20 + (n-1)*var(y) + (k0*n/kn)*(ybar - mu0)^2)/vn

  # numerator of pdf
  numerator <- (1/sqrt(2*pi/k0))*((vn*sigma2n/2)^(vn/2))*gamma((vn+1)/2)

  # denominator of pdf
  denominator <- gamma(vn/2)*((kn*(theta-mun)^2 +
                                vn*sigma2n)/2)^((vn+1)/2)

  # kernel part of the pdf
  `P(theta|y)` <- numerator/denominator

}
# function ends

# hyper-parameter values (these were arbitrarily chosen)
k0 <- 20
v0 <- 7
mu0 <- 5.64
sigma20 <- 27.43

# sample data
y <- rnorm(n = 50, mean = 9, sd = 7)
n <- length(y)
ybar <- mean(y)

# updated parameters (based on hyper-parameters and data)
kn <- k0 + n
vn <- v0 + n
mun <- (k0*mu0 + n*ybar)/kn
sigma2n <- (v0*sigma20 + (n-1)*var(y) + (k0*n/kn)*(ybar - mu0)^2)/vn

set.seed(27)
# sample of variances from the inverse-gamma marginal posterior
`sigma^2|y` <- 1/rgamma(n = 10000,
                        shape = vn/2,
                        rate = vn*sigma2n/2)

# Monte-Carlo Approximation for marginal posterior of theta
`theta|sigma^2,y` <- sapply(
  X = `sigma^2|y`,
  FUN = function(variance){
    rnorm(n = 1,
          mean = mun,
          sd = sqrt(variance))
  }
)

# Monte-Carlo Approximated Density (visuals)
```

```r
tibble(theta.post = `theta|sigma^2,y`) %>%
  ggplot(aes(x = theta.post)) +
  # 1st layer
  geom_histogram(aes(y = after_stat(density)),
                 color = "darkblue",
                 fill = "lightblue") +
  # 2nd layer
  # We're implementing Discrete Approximation here.
  geom_line(data = tibble(
    theta.values = seq(-20, 40, length.out = 500),
    # unmarginalized densities below
    theta.density = sapply(
      X = theta.values,
      FUN = function(val) {
        # set.seed(27) # to reproduce the same sample
        mp.theta(theta = val, y = y)
      }
    )
  ),
  aes(
    x = theta.values,
    y = theta.density/sum(theta.density) # marginalized densities here
  ),
  color = "red",
  linewidth = 1)
```

```
## `stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```



Conlcusion

The discrete-approximated marginal posterior of θ doesn't match the Monte-Carlo estimate of its density graph.

Ex-5.4

There is not a single proper discussion on the Jeffrey's prior in this textbook. I'll come back to this exercise later if possible but I don't think it's all that important.

Ex-5.5

I don't know what a unit information prior is.